Smart Energy Grid Analytics

Advanced Big Data Analytics – Assignment I

Ingestion, storage benchmarking, analysis, dashboards and forecasting for 4,000 smart meters

TEAM DETAILS

101406 Pacifique Mashami
101368 Prince Murenzi
101371 Delice Sebaganwa Shami

Module: BIG DATA ANALYTICS

Instructor: Temitope Oguntade

Date of Submission: 15 October 2026

Repository: `https://github.com/mashami/advance_bigdata_analysis_assignment1`

Contents

Overview **2**

1 Build the ingestion pipeline **2**
1.1 Confirm the environment . . . 2
1.2 Create the database structures . . . 3
1.3 Capture and publish a pilot stream . . . 6

2 Capture and prepare the historical dataset **8**
2.1 Capture and load the complete stream . . . 8
2.2 Investigate data quality . . . 9
2.3 Clean and preserve a consistent benchmark . . . 10

3 Compare chunk intervals **11**
3.1 Create and verify three configurations . . . 11
3.2 Implement and time the five workloads . . . 11
3.3 Explain the chunk results . . . 12

4 Compare compression strategies **13**
4.1 Create the two compressed configurations . . . 13
4.2 Measure storage and compression cost . . . 13
4.3 Repeat the query comparison . . . 14
4.4 Select a storage configuration . . . 14

5 Investigate the utility data **14**
5.1 Identify demand patterns . . . 14
5.2 Compare regions and customer types . . . 16
5.3 Assess reporting quality . . . 16
5.4 Investigate abnormal consumption . . . 17
5.5 Management recommendation . . . 18

6 Create and investigate continuous aggregates **18**
6.1 Create the three aggregate views . . . 18
6.2 Test freshness and delayed arrivals . . . 19

7 Build and compare dashboards **19**
7.1 Raw dashboard . . . 19
7.2 Aggregate dashboard . . . 20
7.3 Verify matching results . . . 20
7.4 Measure performance and judge the tradeoff . . . 20

8 Forecast the next day **21**
8.1 Prepare chronological data and features . . . 21
8.2 Evaluate the previous-day baseline . . . 22
8.3 Train and evaluate one regression model . . . 22
8.4 Judge forecast usefulness . . . 22

9 Reproducibility and contributions **24**
9.1 Code and repository . . . 24
9.2 Contributions . . . 24

Overview

We acted as data analysts for an electricity utility. The supplied, unchanged generator `smart_meter_simulator.py` (v3.0) produced 15-minute readings from 4,000 meters for 1–28 September 2026. We built an MQTT and bulk ingestion pipeline into PostgreSQL/TimescaleDB, investigated and cleaned the data, benchmarked chunk intervals and compression strategies, analysed demand, built continuous aggregates and two dashboards, and forecast next-day hourly regional consumption. All times are CAT (Africa/Kigali, UTC+2). All code is our own except the generator; the execution order is in the repository `README.md`.

Area	Choice	Result
Ingestion	generator → CSV → `COPY`	10,696,848 rows in 194.6 s
Storage	1-day chunks, compressed, segmented by `meter_id`	64.6 % smaller
Dashboards	continuous aggregates	6.7× faster refresh
Anomalies	same-meter, same-slot median ratio rule	40/40 events, 0 false flags
Forecast	Ridge regression	MAE 4.56 vs 42.71 kWh baseline

Table 1: Summary of decisions and headline results.

1 Build the ingestion pipeline

1.1 Confirm the environment

EMQX and PostgreSQL with TimescaleDB run as Docker containers (`docker-compose.yml`). We created the database `energy_grid`, enabled TimescaleDB and set `ALTER DATABASE energy_grid SET timezone TO 'Africa/Kigali'`, so every session works in CAT. Daily buckets use `time_bucket('1 day', ts, 'Africa/Kigali')`, which aligns them to CAT midnight. The details below were recorded by `src/s1_1_environment.py`.

Item	Value
CPU	Apple M3, 8 cores
RAM	16 GB
Operating system	macOS 14.6
Storage	SSD (Apple Fabric)
Python	3.13.7
EMQX	6.3.1
PostgreSQL	17.11
TimescaleDB	2.30.2
`shared_buffers`	1959 MB
`work_mem`	7837 kB
`max_parallel_workers_per_gather`	4
`jit`	off
PostgreSQL runtime	Docker container (`timescale/timescaledb:latest-pg17`) inside the Docker Desktop Linux VM. VM limit: 8 CPUs and 7.65 GiB RAM. No per-container CPU or RAM limit.
Session time zone	Africa/Kigali

Table 2: Environment.

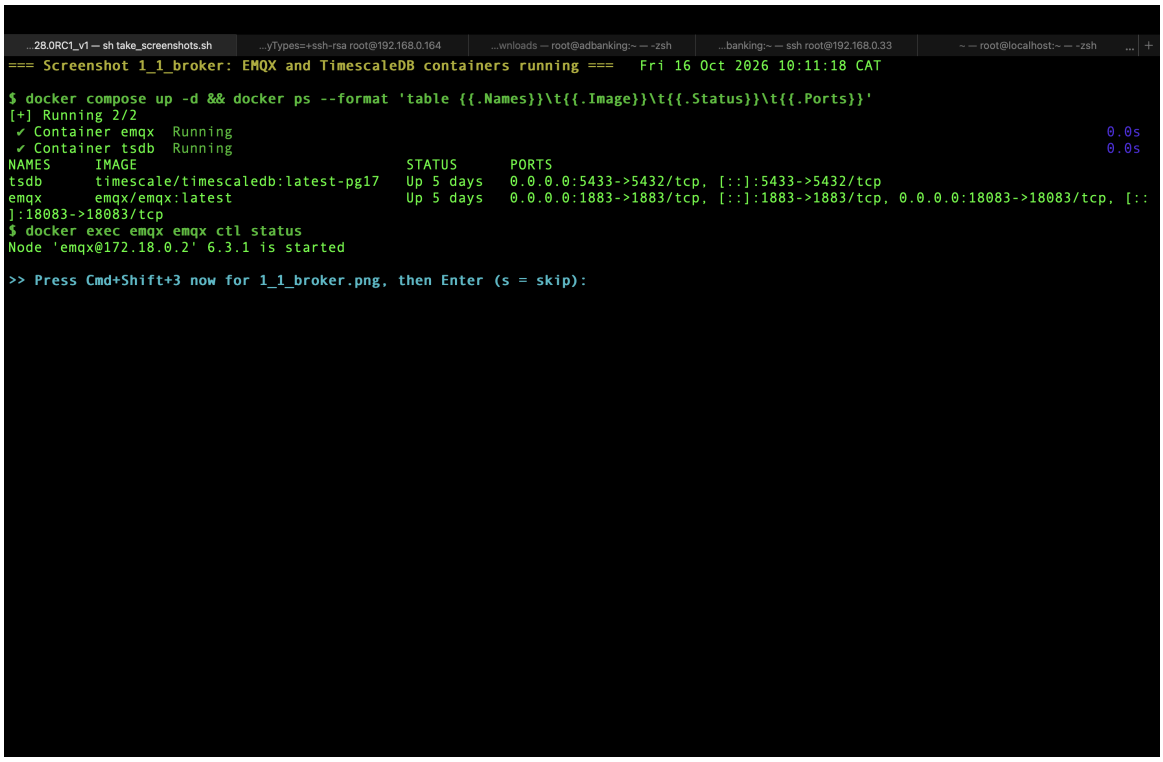


Figure 1: EMQX broker running (`docker ps` / EMQX dashboard).

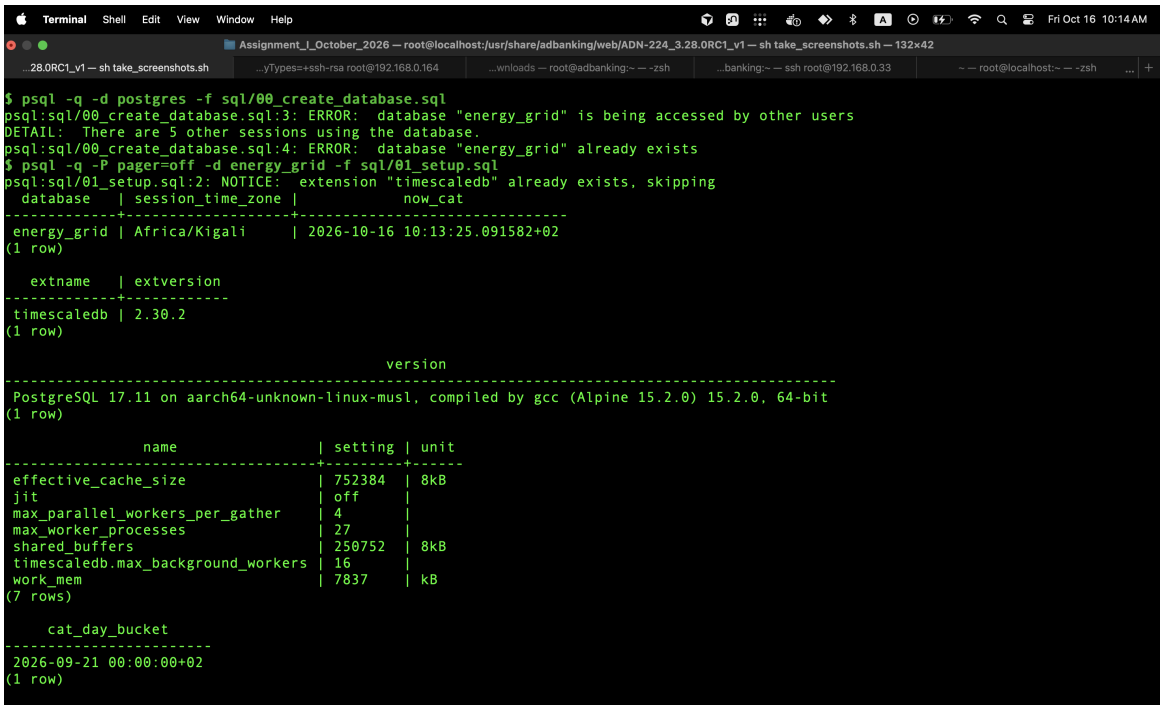


Figure 2: Connection to `energy_grid`, the TimescaleDB extension and the session time zone.

1.2 Create the database structures

`sql/02_tables.sql` creates three tables:

- `energy_live`, a regular table for the pilot;
- `energy_readings`, a hypertable on `event_time` with one-day chunks and no primary key, so

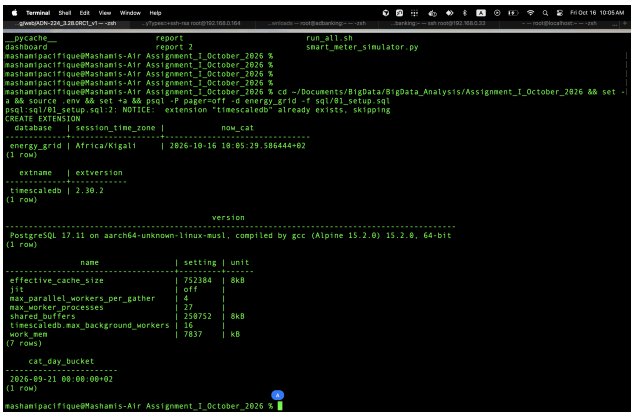


Figure 3: Enter Caption

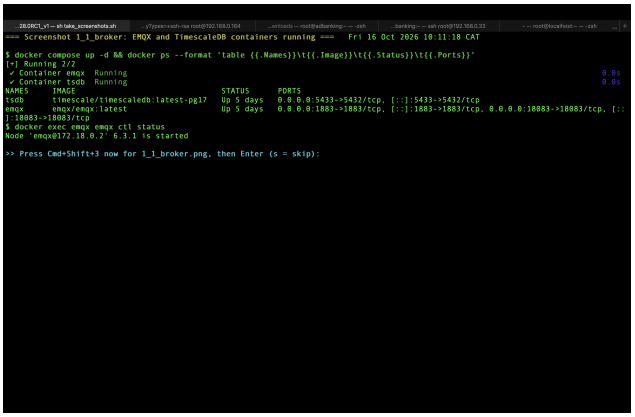


Figure 4: Enter Caption

duplicates are allowed;

- `meters`.

The hypertable is created without default indexes and receives two explicit B-tree indexes, `(event_time DESC)` and `(meter_id, event_time DESC)`, so the same indexes can be reproduced on every later hypertable. `src/s1_2_load_meters.py` inserted the 4,000 `meter_metadata()` dictionaries, and the count query confirmed 4,000 rows.

Table	Column	Type	Unit	Meaning
energy_live, energy_readings	meter_id	BIGINT	–	ten-digit meter identifier
	event_time	TIMESTAMPTZ	CAT time	start of the 15-minute interval
	received_time	TIMESTAMPTZ	CAT time	simulated arrival (not insert time)
	power_kw	DOUBLE PRECISION	kW	average power over the interval
	voltage_v	DOUBLE PRECISION	V	voltage
	current_a	DOUBLE PRECISION	A	current (power factor 0.95)
	frequency_hz	DOUBLE PRECISION	Hz	grid frequency
	energy_kwh	DOUBLE PRECISION	kWh	interval energy ≈ power_kw × 0.25 h
meters	meter_id	BIGINT (PK)	–	meter identifier
	region	TEXT	–	Region 1–10 (400 meters each)
	customer_type	TEXT	–	residential, commercial, industrial
	base_power_kw	DOUBLE PRECISION	kW	typical power level of the meter
	power_factor	DOUBLE PRECISION	none	real/apparent power ratio (0.95)

Table 3: Data dictionary.

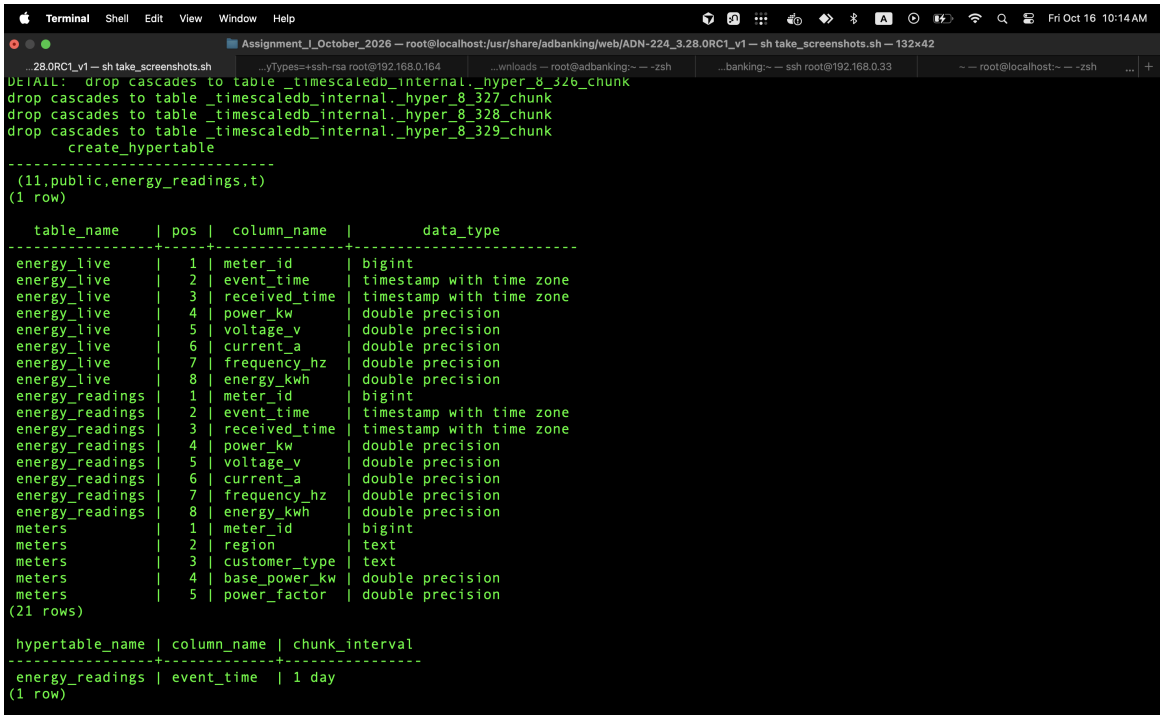


Figure 9: Columns and types of the three tables, and the one-day chunk interval.

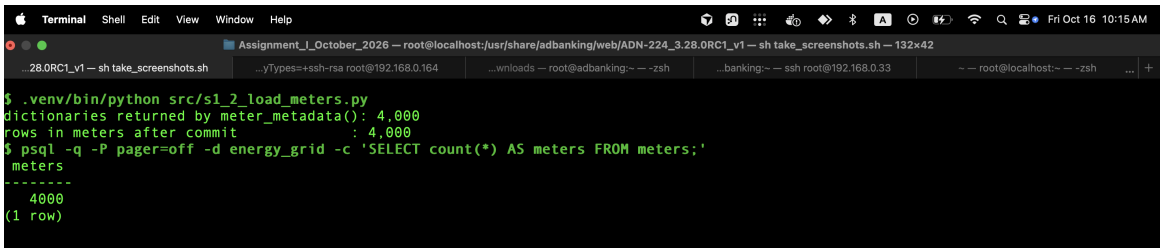


Figure 10: Query confirming 4,000 rows in `meters`.

1.3 Capture and publish a pilot stream

The publisher (`src/s1_3_publisher.py`) takes the first 2,000 generator dictionaries and publishes each as JSON to `energy/meters/{meter_id}` with QoS 1 and no retained messages. A message counts as acknowledged when the broker's PUBACK triggers `on_publish`. The subscriber (`src/s1_3_subscriber.py`) is started first on an empty `energy_live`. It subscribes to `energy/meters/#` at QoS 1 with a persistent session and inserts the eight fields of each message.

Stage	Run 1 (faulty)	Run 2 (fixed)
Dictionaries taken from the generator	2,000	2,000
Messages acknowledged by the broker	2,000	2,000
Messages received by the subscriber	**1,208**	2,000
Rows committed in `energy_live`	**1,208**	2,000

Table 4: Pilot counts. Deliberate generator duplicates count as separate messages: the 2,000 rows contain 1,994 distinct pairs.

Fault diagnosis.

- **Size.** The first run lost 792 messages (39.6 %) between the broker and the subscriber. Nothing was lost between the generator and the broker.
- **Evidence.** The subscriber log showed no parse errors, no failed inserts and no redeliveries. The broker metrics (`emqx ctl broker metrics`) reported `delivery.dropped.queue_full` = 792.
- **Cause.** Version 1 inserted and committed each row inside the MQTT `on_message` callback, which blocks the client's network loop. The subscriber therefore drained about 200 messages per second, while the publisher sent all 2,000 in about 1.4 s. EMQX holds at most 1,000 queued messages per session and discards new ones once that queue is full, even at QoS 1.
- **Fix.** The callback now only parses each message and places it on an in-process queue; a separate writer thread inserts batches of 200 rows and commits. The repeated pilot delivered 2,000 at every stage.

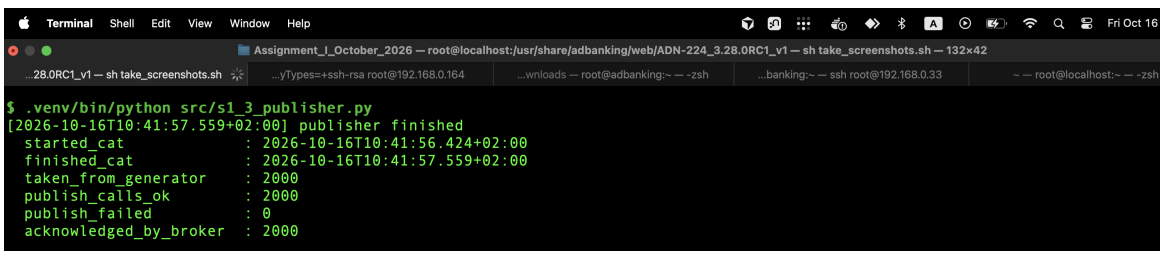


Figure 11: Publisher completion with 2,000 acknowledged messages.

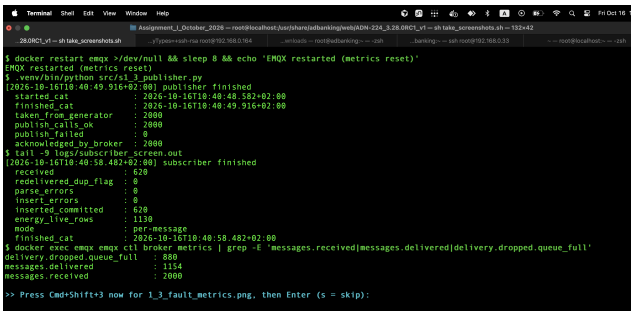


Figure 12: Enter Caption

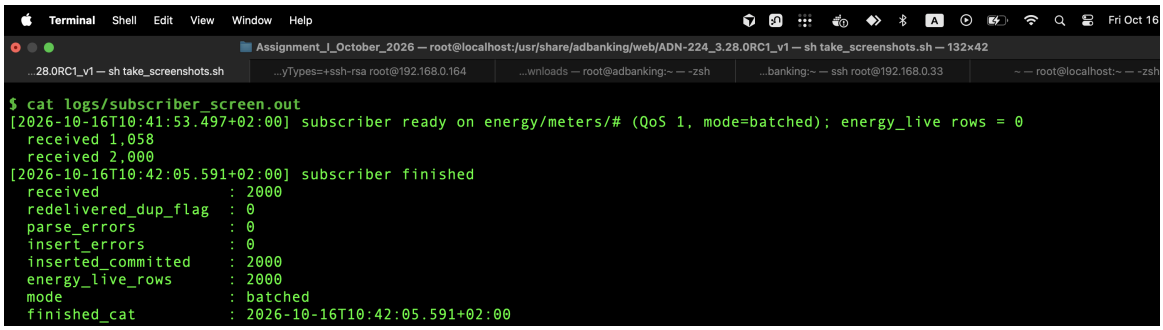


Figure 21: Subscriber received count.

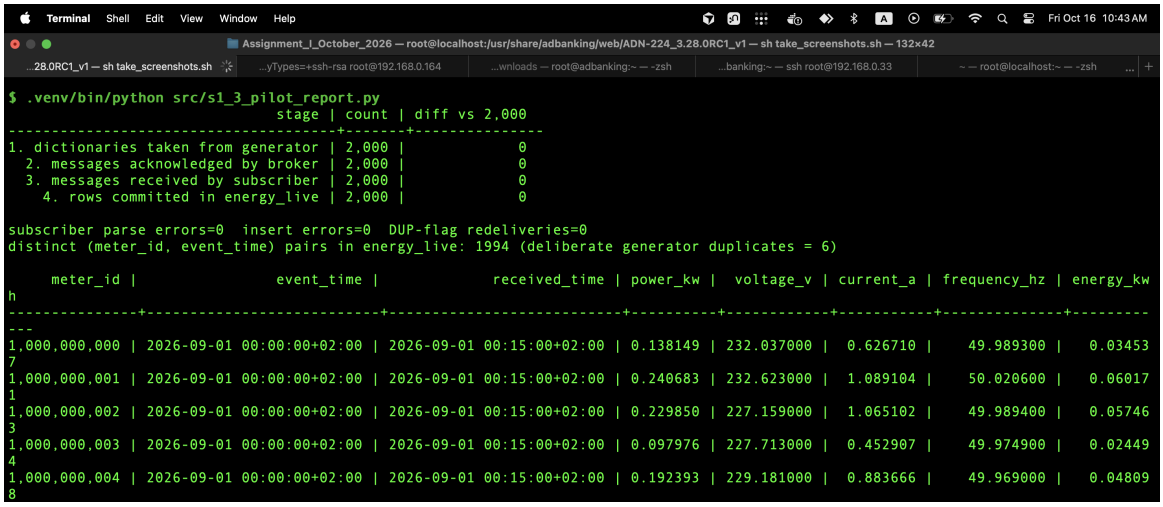


Figure 22: Stored count in `energy_live` and sample rows.

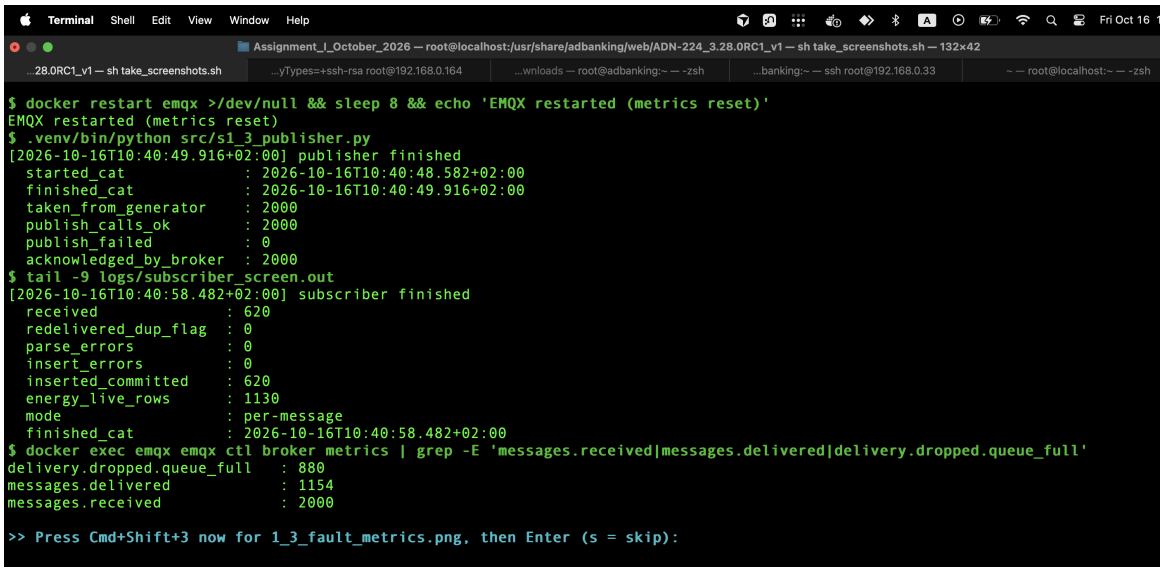


Figure 23: First run: EMQX `delivery.dropped.queue_full = 792`.

2 Capture and prepare the historical dataset

2.1 Capture and load the complete stream

Route: generator → CSV file → `COPY FROM STDIN` (`src/s2_1_capture_load.py`). Readings are written to the CSV one dictionary at a time, with progress printed every million rows, so memory use stays constant. The file is then streamed into the empty `energy_readings` in 8 MB blocks inside a single transaction. `dataset_info()` gave `planned_readings` = 10,752,000 and `emitted_readings` = 10,696,848.

Measure	Value
Dictionaries consumed	10,696,848
CSV data rows (1.16 GB)	10,696,848
Committed rows in `energy_readings`	10,696,848
CSV generation	72.9 s
Database load (first write to final commit)	120.2 s
Total (start of generation to final commit)	194.6 s
Distinct meters	4,000
Earliest / latest `event_time`	2026-09-01 00:00 / 2026-09-28 23:45
Table data / indexes / total (29 chunks)	958 MB / 466 MB / 1,424 MB

Table 5: Full capture and load.

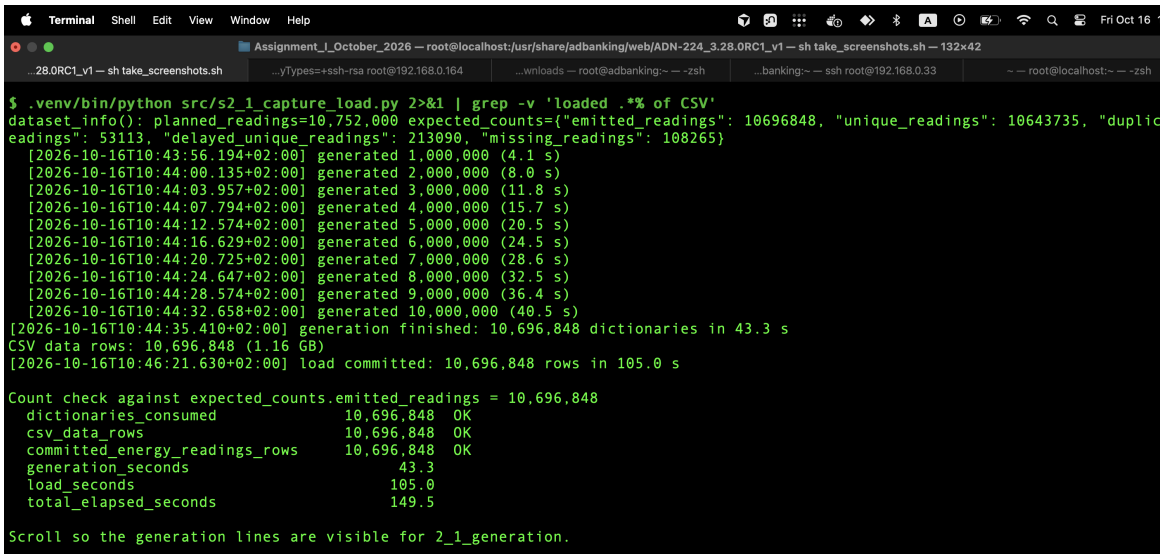


Figure 24: Completed generation with progress and final consumed count.

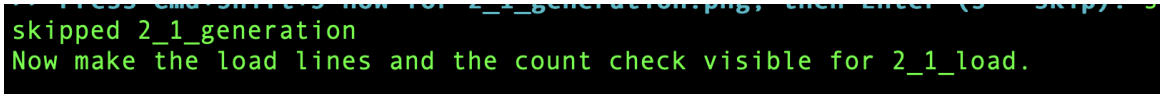


Figure 25: Completed load and count comparison.

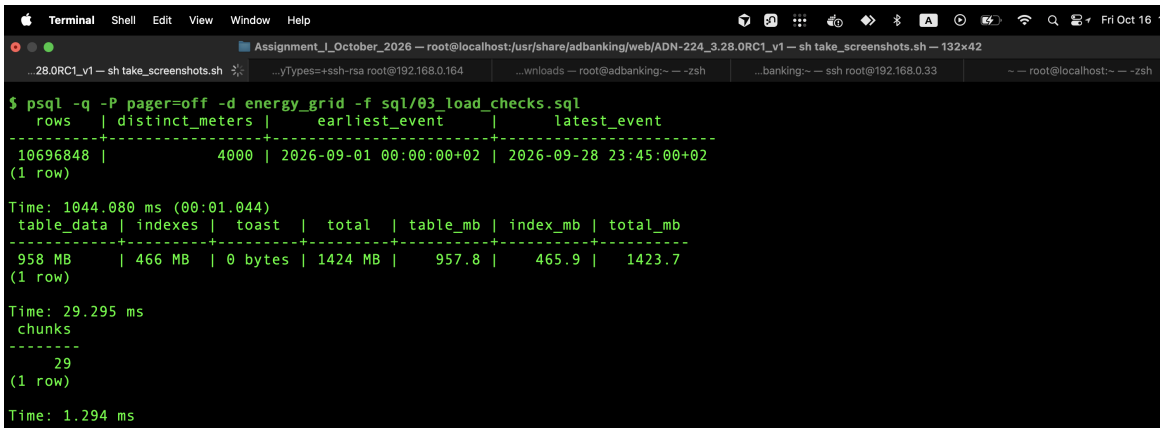


Figure 26: Distinct meters, date range and table, index and total size.

2.2 Investigate data quality

A unique reading is one `(meter_id, event_time)` pair. `sql/04_quality.sql` measures each count as follows:

- **Missing pairs:** anti-join of the planned grid (4,000 meters × 2,688 slots) against the observed pairs.
- **Extra copies:** $\sum(\text{copies} - 1)$ over observed pairs.
- **Delayed readings:** unique pairs with `received_time` later than `event_time` + 15 minutes.

Entry	Measured	Expected	Match
planned_readings	10,752,000	10,752,000	
emitted_readings	10,696,848	10,696,848	
unique_readings	10,643,735	10,643,735	
duplicate_readings	53,113	53,113	
delayed_unique_readings	213,090	213,090	
missing_readings	108,265	108,265	

Table 6: Measured against reference counts.

Rates and identity checks.

- Missing 1.0069 %, duplicates 0.4990 %, delayed 2.0020 %, completeness 98.9931 %.
- Unique + extra copies = 10,643,735 + 53,113 = 10,696,848, which equals the emitted count.
- Planned − unique = 108,265, which equals the missing count.
- On-time readings arrive exactly 15 minutes after `event_time`; delayed readings arrive 30–180 minutes after.

Meter	Region	Type	Missing
1000003622	Region 3	commercial	47
1000000106	Region 7	industrial	45
1000000366	Region 7	commercial	45
1000001259	Region 10	residential	44
1000000189	Region 10	commercial	43

Table 7: Five meters with the most missing readings (average about 27 per meter).

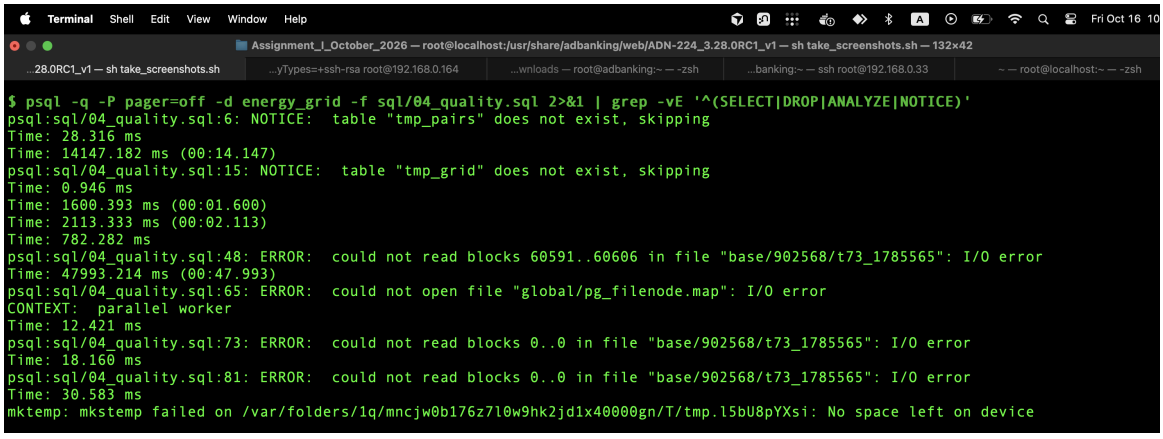


Figure 27: Diagnostic queries and their results.

2.3 Clean and preserve a consistent benchmark

`sql/05_clean.sql` cleans the table in three steps:

1. Group rows by `(meter_id, event_time)` and count the distinct versions of the six value columns. This found 53,113 duplicate pairs and **0 conflicting** ones.
2. Delete every copy after the first among rows identical in all eight columns, identifying each physical row by `(tableoid, ctid)`. Only pairs whose copies agree are touched, so a conflicting pair could never be deleted silently.

3. Run `VACUUM ANALYZE`.

	Rows	$\sum$ energy_kwh
Before cleaning	10,696,848	3,872,863.893
After cleaning	10,643,735	3,853,704.347

Table 8: Cleaning results. No duplicate pairs remain; 19,159.5 kWh of double-counted energy was removed.

Why missing readings must not become zero. A missing reading means the consumption is unknown, not that it was zero. Filling gaps with zero would:

- understate energy totals and bills;
- create false "unusual drop" anomalies;
- mislead the forecast model;
- hide the true completeness of the data.

Gaps are therefore kept as gaps and reported through completeness. Any estimate must be explicit and labelled as such.

3 Compare chunk intervals

3.1 Create and verify three configurations

`energy_readings_3h` and `energy_readings_week` have the same eight columns and the same two B-tree indexes as `energy_readings`, and hold copies of the cleaned rows (`sql/06_chunk_tables.sql`). TimescaleDB aligns chunk boundaries to the Unix epoch in UTC. One-day chunks therefore run from 02:00 to 02:00 CAT, and weekly chunks start on Thursdays.

Table	Interval	Chunks	First chunk start	Last chunk end
energy_readings_3h	3 hours	225	2026-08-31 23:00	2026-09-29 02:00
energy_readings	1 day	29	2026-08-31 02:00	2026-09-29 02:00
energy_readings_week	7 days	5	2026-08-27 02:00	2026-10-01 02:00

Table 9: Chunk configurations.

All three tables match on every check:

- 10,643,735 rows and 4,000 meters;
- first and last `event_time`: 2026-09-01 00:00 and 2026-09-28 23:45;
- $\sum$ power_kw = 15,414,817.388 and $\sum$ energy_kwh = 3,853,704.347;
- the largest difference between tables is 9×10^{-9}, well within the 0.001 tolerance.

3.2 Implement and time the five workloads

The five queries are kept in `sql/08_workloads.sql` with the table name as the only parameter. Q2 orders by total descending and then `event_time` ascending before `LIMIT 10`, so a tie at tenth place still returns exactly ten rows.

Measurement method.

- **Client:** psycopg 3 (Python), one connection on machine `Mashamis-Air`.
- **Timer:** `time.perf_counter()`, from statement submission until the last row is fetched.
- **Runs:** after `ANALYZE`, one warm-up and five measured runs per query and table, executed sequentially with no restarts.
- **Results:** identical across the three tables within 0.001.
- **Session:** 10:22:17 to 10:23:09 CAT on 3 October, 51.7 s.

All values are **warm-cache** measurements.

	3 hours			1 day			1 week		
Query	median	min	max	median	min	max	median	min	max
Q1	**39.4**	28.6	47.3	55.7	33.7	351.8	55.5	50.1	76.3
Q2	152.1	148.1	205.7	**137.8**	127.6	143.1	169.4	162.4	204.7
Q3	2196.4	1912.4	4012.2	842.2	679.6	1829.6	**641.0**	591.7	1410.8
Q4	2.96	2.65	4.00	1.28	1.15	1.70	**0.96**	0.84	2.10
Q5	546.5	496.7	2221.5	502.8	423.5	598.2	**458.5**	398.0	1111.6

Table 10: Warm-cache timings in ms (five measured runs after one warm-up). Bold marks the fastest median.

3.3 Explain the chunk results

Chunks opened per query were taken from the `EXPLAIN (ANALYZE, BUFFERS)` plans:

Query	3 hours	1 day	1 week
Q1 (one CAT day)	9	2	1
Q2, Q4 (seven CAT days)	57	8	2
Q3, Q5 (all data)	225	29	5

Table 11: Chunks opened per query.

Q1: 3-hour chunks fastest.

- *Observation:* 39 ms against about 55 ms for the other two tables.
- *Explanation:* the 9 small chunks cover the CAT day almost exactly. The daily table opens 2 chunks, because the day crosses the 02:00 boundary, and discards about half their rows. The weekly table searches one chunk seven times larger.

Q2: 1-day chunks fastest.

- *Observation:* 138 ms, against 152 ms for 3 hours and 169 ms for a week.
- *Explanation:* the week maps onto 8 chunks with little waste. With 3-hour chunks, 57 chunks add planning and executor overhead. With weekly chunks, rows outside the window must be filtered out.

Q3 and Q5: weekly chunks fastest.

- *Observation:* Q3 took 641 ms (against 842 and 2,196 ms); Q5 took 458 ms (against 503 and 547 ms).

- *Explanation:* with no time filter, every row is read and chunk exclusion cannot help. The cost difference is per-chunk overhead: one scan node and one partial aggregate per chunk. Q3 on 225 chunks must merge 225 partial groupings of 4,000 meters, so it is 2.6× slower than the daily table.

Q4: weekly chunks fastest.

- *Observation:* 0.96 ms, against 1.28 and 2.96 ms.
- *Explanation:* the `(meter_id, event_time)` index probe costs roughly the same in each chunk, so time grows with the number of chunks: 2, 8 or 57.

Conclusion. Small chunks only help short, time-filtered queries; large chunks help whole-history scans. One-day chunks are the most balanced choice.

4 Compare compression strategies

4.1 Create the two compressed configurations

We chose **one-day chunks**, following Section 3.3. `src/s4_1_compress.py` builds the two tables as follows:

1. Record one cutoff, the experiment start minus 48 hours: 2026-10-01 10:24:41 CAT. The same cutoff is used for both tables.
2. Create `energy_time_compressed` and `energy_meter_compressed` with the same columns and indexes. Both are ordered by `event_time DESC`. Segmentation is set to `''` (none) for the time table and `meter_id` for the meter table.
3. Add a 48-hour compression policy to each table and pause it at once with `alter_job(..., scheduled => false)`.
4. Copy the cleaned data one chunk at a time. After each chunk is fully loaded, run `compress_chunk()` if its end precedes the cutoff, timing loading and compression separately.

All 29 chunk ranges ended before the cutoff, so both tables compressed the same 29 ranges. The data checks from Section 3.1 match: 10,643,735 rows, 4,000 meters, and equal dates and sums.

4.2 Measure storage and compression cost

Sizes come from `hypertable_detailed_size`, which includes every chunk, compressed or not, its TOAST storage and its indexes.

Table	Data MB	Index MB	Total MB	Ratio	Saved %	Compr. time s	Chunks compr.	Rows %
energy_readings (baseline)	950.9	465.9	1416.7	1.00	–	–	0/29	0
energy_time_compressed	437.8	0.9	438.7	**3.23**	**69.0**	11.5	29/29	100
energy_meter_compressed	496.2	5.5	501.7	2.82	64.6	30.4	29/29	100

Table 12: Storage and compression cost. Ratio = baseline total / compressed total.

Segmenting by meter compresses less and takes 2.6× longer. A one-day chunk holds only 96 readings per meter, so segmentation produces 4,000 small batches per chunk instead of about 380 full 1,000-row batches. Each batch carries its own metadata and column headers.

4.3 Repeat the query comparison

The baseline is `energy_readings` (uncompressed, one-day chunks), timed in the same session as the compressed tables (10:31:18–10:31:34 CAT, 16.0 s). Improvement is 100×(baseline−compressed)/baseline; positive values mean faster. All results matched within 0.001.

Query	3 h	1 day	week	time-compr.	meter-compr.	time impr. %	meter impr. %
Q1	39.4	27.1	55.5	**18.7**	232.9	+31.1	−758.3
Q2	152.1	116.0	169.4	95.9	**60.7**	+17.3	+47.7
Q3	2196.4	676.8	641.0	148.9	**123.6**	+78.0	+81.7
Q4	2.96	1.68	0.96	27.9	1.67	−1562.8	+0.3
Q5	546.5	499.9	458.5	**139.6**	169.7	+72.1	+66.0

Table 13: All five configurations: median ms, warm cache.

After the measurements, the policies were resumed (`src/jobs.py resume`).

4.4 Select a storage configuration

Effect of meter segmentation.

- **Single meter (Q4).** Segmentation keeps the query as fast as uncompressed (1.67 ms), because only that meter's batches are decompressed. Without segmentation every batch of the week must be decompressed, which is 17× slower (27.9 ms).
- **All meters in a short time window (Q1).** The effect reverses: one CAT day spans 4,000 small meter batches per chunk (233 ms). Time ordering skips whole batches using their minimum and maximum `event_time` (18.7 ms).
- **Full-history scans (Q3, Q5).** Both strategies gain 66–82 %, because far fewer bytes are read.

Recommendation.

1. **Choice:** one-day chunks, compressed after 48 hours, segmented by `meter_id` and ordered by `event_time DESC`.
2. **Evidence:**
   - storage falls from 1,417 MB to 502 MB (ratio 2.82, 64.6 % saved);
   - Q2, Q3 and Q5 run 48–82 % faster;
   - single-meter lookups, used for billing and customer service, stay at 1.7 ms instead of 28 ms.
3. **Tradeoff:**
   - compression takes 30.4 s against 11.5 s, and saves 4.4 points less storage than time-only ordering;
   - all-meter, short-window queries such as Q1 become slow, so they should be served from the continuous aggregates of Section 6.

5 Investigate the utility data

5.1 Identify demand patterns

Weekdays used about 141–152 MWh per day and weekends about 112–119 MWh, roughly 21 % less. Consumption also grew by about 2 % each week.

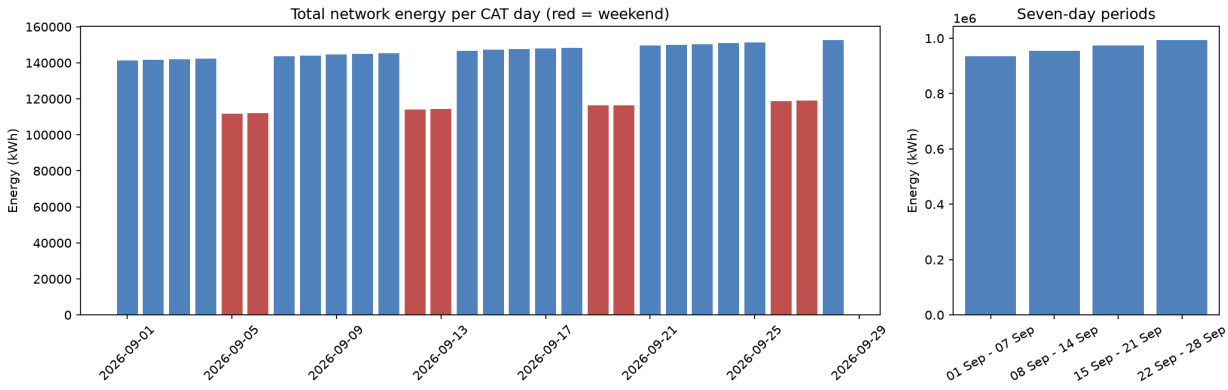


Figure 28: Total network energy per CAT day (weekends in red) and per seven-day period.

Seven-day period	Energy (kWh)	Change
1–7 Sep	934,199.1	–
8–14 Sep	953,715.0	+2.1 %
15–21 Sep	973,230.0	+2.0 %
22–28 Sep	992,560.3	+2.0 %

15-minute interval (CAT)	Network kW
Mon 28 Sep 13:00	7,993.7
Mon 28 Sep 12:45	7,943.9
Mon 28 Sep 13:15	7,943.6

Table 14: Weekly energy totals (left) and the three highest demand intervals (right).

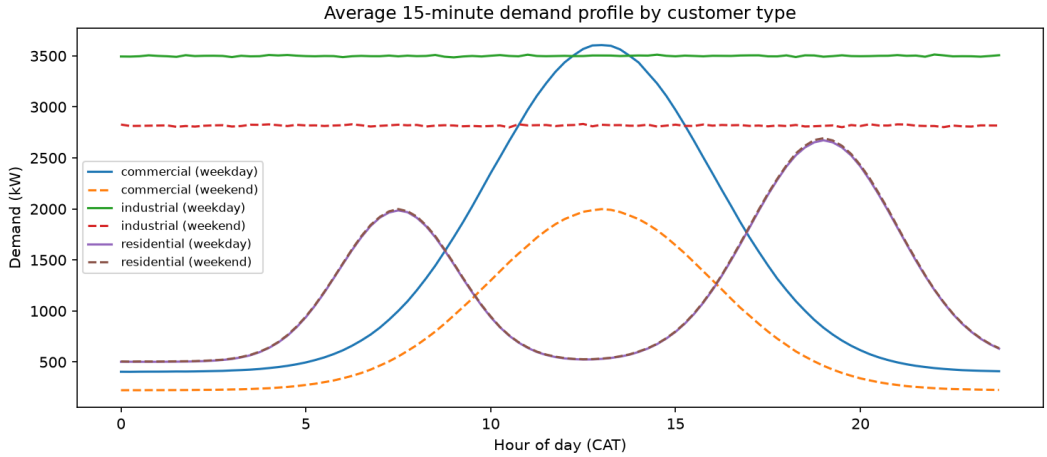


Figure 29: Average 15-minute demand profile by customer type and day type.

Patterns.

- Industrial meters provide a large, almost flat base load.
- Commercial load peaks around 13:00 on business days and falls at weekends.
- Residential load has a morning peak (about 07:30) and an evening peak (about 19:00).

Network demand therefore peaks at weekday midday. The three highest intervals fall on the last Monday because of the growth trend.

Operational implications.

- Capacity must cover a growing weekday midday peak.
- Maintenance is best scheduled at night or at weekends.

- Energy procurement needs to grow by about 2 % per week.

kW versus kWh. Kilowatts (kW) measure instantaneous demand, which sizes generation and network capacity. Kilowatt-hours (kWh) measure energy delivered over time, which drives billing and procurement. A short, high kW peak can be costly even when its kWh contribution is small.

5.2 Compare regions and customer types

Customer type	Meters	Energy (kWh)	kWh per meter	Share
industrial	406	2,220,672	5,469.6	57.6 %
commercial	1,010	825,475	817.3	21.4 %
residential	2,584	807,557	312.5	21.0 %

Table 15: Energy by customer type over 28 days.

Region	Meters	Energy (kWh)	kWh per meter	Industrial meters
Region 8	400	503,475.8	1,258.7	61
Region 5	400	411,040.4	1,027.6	48
Region 9	400	400,467.2	1,001.2	45
Region 3	400	392,376.6	980.9	46
Region 1	400	383,539.8	958.8	37
Region 2	400	378,448.9	946.1	39
Region 7	400	362,288.5	905.7	35
Region 10	400	351,149.8	877.9	34
Region 6	400	346,512.8	866.3	32
Region 4	400	324,404.5	811.0	29

Table 16: Energy by region over 28 days, with the number of industrial meters.

Customer types. Normalising by meter count removes the effect of group size. An industrial meter still uses 17.5× as much energy as a residential one.

Regions. Every region has 400 meters, so normalisation does not change the ranking: Region 8 still uses 55 % more energy per meter than Region 4. The industrial meter count follows the same ranking (61 against 29). Regional differences therefore reflect customer mix, not regional behaviour. Capacity planning should follow the number of industrial customers, and fair regional comparisons also need normalisation by customer type.

5.3 Assess reporting quality

Completeness. Over the whole network, completeness was 98.993 %. Regions ranged from 98.969 % (Region 6) to 99.013 % (Region 8), a spread of only 0.04 points. Losses are random, not a regional communication fault.

Energy for 21 September	kWh	Readings
Arrivals by 22 Sep 00:00	149,400.54	379,505
All arrivals	149,552.72	380,036
Difference	152.18 (0.102 % of final)	531

Table 17: Effect of delayed arrivals on a daily total.

Effect of delayed and missing readings.

- **Delayed readings.** All 531 late readings came from the last three hours of the day. A report produced at midnight understates the evening peak, but the data arrives later, so daily totals should be marked provisional and finalised after about 03:00.
- **Missing readings.** The roughly 1 % of readings that never arrive is never recovered, so totals are permanently about 1 % low unless gaps are estimated explicitly.

5.4 Investigate abnormal consumption

Rule. The rule was fixed before the labels were consulted. For each reading:

- the *expected* power is the median `power_kw` of the same meter, in the same 15-minute slot of the CAT day and the same day type (weekday or weekend), over all 28 days;
- the reading is flagged when power/expected ≥ 2.0 (spike) or ≤ 0.4 (drop).

Reasons for this rule.

- Each customer type has its own daily shape, and commercial load falls at weekends. Comparing like with like removes the normal pattern.
- The median is not moved by a single abnormal day.
- Normal noise is about ± 15 % plus at most 8 % trend, so normal ratios stay within roughly 0.8–1.25. The 2.0 and 0.4 limits leave a wide margin.

The flags were written to the table `anomaly_flags` (`meter_id`, `event_time`). Only then did the program call `anomaly_events()`.

Measure	Value
Flags raised (spike / drop)	474 (236 / 238)
Injected events detected	**40 / 40**
Injected events missed	0
False flags	0 (0 % of 10,643,261 readings outside injected intervals)
Percentage of meters affected by false flags	0 % (0 of 4,000)

Table 18: Anomaly detection evaluated against the 40 supplied events.

Interpretation. The 474 flags cover every observed reading inside the 40 three-hour windows (480 planned, 6 missing). The injected factors of 2.8× and 0.03× lie far outside normal variation, which is why nothing was missed and nothing was falsely flagged. In real data:

- events would be missed when changes are milder, when they fall inside missing readings, or when an anomaly recurs often enough to shift the median;
- false alarms would come from holidays, outages or genuine changes in a customer's equipment.

Lowering the thresholds trades more detections for more false alarms.

5.5 Management recommendation

To: Operations and IT management **Re:** Smart-meter analytics, September 2026

Operations. Network demand peaks on weekday middays (about 13:00), driven by commercial load on top of a large, flat industrial base. The highest 15-minute demand was 7,994 kW on Monday 28 September, and weekly energy grew steadily by about 2 % per week (934 to 993 MWh). Capacity plans should therefore be based on the weekday midday kW peak, adjusted for this growth. Maintenance should be scheduled at weekends, when consumption is about 21 % lower. Industrial meters (10 % of meters) use 58 % of the energy, and they explain why Region 8 uses 55 % more per meter than Region 4. Key-account monitoring for industrial customers would give the largest benefit per meter. Our anomaly rule, which compares each reading with the same meter's usual load at that time and day type, found all 40 test events with no false alarms. We recommend running it daily and investigating flags within 24 hours.

Data quality and reporting. About 1 % of readings never arrive and 2 % arrive up to three hours late. Daily reports produced at midnight understate consumption (by 152 kWh, 0.1 %, on 21 September). Gaps must never be treated as zero use. Daily figures should be published as provisional at midnight and finalised after 03:00, with completeness (currently 98.99 %, uniform across regions) reported alongside.

Storage and dashboards. Keep raw readings in TimescaleDB hypertables with one-day chunks, compressed after 48 hours with meter segmentation. This cuts storage by 65 % (1,417 to 502 MB per month of data). Full-history queries become 66–82 % faster and single-meter lookups stay at about 2 ms. Serve dashboards from continuous aggregates: a full refresh takes 0.54 s instead of 3.6 s (6.7× faster), for 18 MB of extra storage and a 15-minute refresh job.

Forecasting. Adopt the Ridge regression day-ahead forecast. On the test week its error was 4.56 kWh MAE per region-hour (RMSE 5.79 kWh), against 42.7 kWh (RMSE 79.4 kWh) for the "same as yesterday" baseline, an 89 % reduction. It is especially better around weekends: on Monday 28 September the baseline under-forecast the regional peak by up to 270 kWh, against at most 18 kWh for the model. Retrain weekly, add holiday information, and run the forecast after late readings have arrived.

6 Create and investigate continuous aggregates

6.1 Create the three aggregate views

`sql/10_caggs.sql` creates three views over `energy_readings`, all `WITH NO DATA` and with `timescaledb.materialized = true`:

- `cagg_energy_15m_region`: 15-minute bucket × region;
- `cagg_energy_hour_region`: CAT hour × region;
- `cagg_energy_day_meter`: CAT day × `meter_id`, using `time_bucket('1 day', ..., 'Africa/Kigali')`.

The two regional views use an inner join to `meters` on `meter_id`. Each view stores the bucket, its key, the sum of energy, the sum of power and the reading count.

`src/s6_1_refresh.py` refreshed each view over 1–29 September and measured it before any policy was added.

View	Rows	Refresh (s)	Data MB	Index MB	Total MB
cagg_energy_15m_region	26,880	15.9	2.11	2.04	4.15
cagg_energy_hour_region	6,720	22.2	0.64	0.59	1.23
cagg_energy_day_meter	112,000	17.6	7.41	5.15	12.55

Table 19: Initial refresh and materialized size.

Policies were then added to all three views: a three-day lookback, a one-hour end offset and a 15-minute schedule. Their window is relative to the database clock, which is now in October, so

they never refresh September. That is why the manual refresh was required.

6.2 Test freshness and delayed arrivals

`src/s6_2_freshness.py` uses its own test hypertable and an hourly aggregate (no data at creation, real-time off, no policy).

- **Test time:** 3 October 10:42 CAT.
- **Hours used:** H = 3 October 09:00, so the older hour is 1 October 09:00.
- **Inserts:** 1 October 09:10 (1 kWh), 1 October 09:25 (0.5 kWh) and 3 October 09:10 (2 kWh).

Stage	Raw older	Raw H	Aggregate older	Aggregate H
1. 1 kWh inserted, older hour refreshed, real-time off	1.0, 1 row	–	1.0, 1 row	no bucket
2. +0.5 kWh older, +2 kWh H, no refresh, real-time off	1.5, 2 rows	2.0, 1 row	1.0, 1 row	no bucket
3. Real-time on, no refresh	1.5, 2 rows	2.0, 1 row	1.0, 1 row	2.0, 1 row
4. Both hours refreshed	1.5, 2 rows	2.0, 1 row	**1.5, 2 rows**	**2.0, 1 row**

Table 20: Freshness test results (kWh, rows).

What became visible at each stage.

- **Stage 1.** Only the refreshed older hour is materialized.
- **Stage 2.** The new rows exist in the raw table but not in the materialization. The older hour is stale at 1 kWh, and no bucket exists for H; an absent bucket is not a zero reading.
- **Stage 3.** Real-time aggregation combines materialized data below the watermark (1 October 10:00) with raw data above it. H therefore appears with 2 kWh, but the late insert into the older hour, which is below the watermark, stays invisible until a refresh processes its invalidation.
- **Stage 4.** The manual refresh brings both hours up to date and moves the watermark to 3 October 10:00.

What the Section 6.1 policy could refresh. At 10:42 the policy's window was [30 September 10:42, 3 October 09:42), and only whole buckets inside the window are refreshed.

- The older hour **could** be refreshed.
- Hour H (09:00–10:00) **could not**: it ends after 09:42, so it becomes refreshable only after 11:00. Until then it is visible only through real-time aggregation.

7 Build and compare dashboards

7.1 Raw dashboard

We used Plotly Dash (`dashboard/app.py`). The Raw dashboard queries `energy_readings` directly and joins `meters` where needed. Its filters are a whole-CAT-day date range and a selected meter. It has five panels:

- hourly regional energy (kWh);
- daily network energy (kWh);
- network energy in the seven-day periods of Section 5.1 (kWh);

- daily energy of the selected meter (kWh);
- completeness by region (%).

Every panel is limited to the selected range. The panel queries are kept in `dashboard/queries.py`.

7.2 Aggregate dashboard

The Aggregate dashboard has the same panels, layout, filters and date range, but reads only the continuous aggregates:

- hourly regional energy comes from `cagg_energy_hour_region`;
- daily and weekly network energy and the observed completeness counts come from `cagg_energy_15m_region` summing the stored sums and counts;
- meter history comes from `cagg_energy_day_meter`.

Planned readings are calculated as registered meters (from `meters`) × days × 96.

7.3 Verify matching results

Both dashboards were set to 22 September 00:00 – 29 September 00:00 (exclusive), with all regions and meter 1000000000. `src/s7_3_verify.py` matched the data behind each panel by its keys.

Panel	Rows compared	Keys and counts identical	Max. abs. difference	Result
Hourly regional energy	1,680	yes	0.000000 kWh	pass
Daily network energy	7	yes	0.000000 kWh	pass
Seven-day network energy	1	yes	0.000000 kWh	pass
Selected meter, daily	7	yes	0.000000 kWh	pass
Completeness by region	10	yes	0.000000 pp	pass

Table 21: Raw against Aggregate panel data (limits: 0.001 kWh and 0.000001 percentage points).

7.4 Measure performance and judge the tradeoff

Procedure.

- All background jobs were paused, and the dashboards have no result caching.
- In each mode, one warm-up and ten measured refreshes ran automatically in the same browser, on the same machine and connection.
- **Tools:** per-panel query time was measured on the server with Python's `time.perf_counter()`. Whole-dashboard time was measured in the browser with `performance.now()`, from the refresh request until all five Plotly graphs had drawn that refresh.

	Raw			Aggregate		
Measure (ms)	median	min	max	median	min	max
Whole dashboard	3586.7	2544.5	4594.4	**536.0**	79.1	577.7
Hourly regional	1874.6	874.1	3871.6	34.3	10.6	39.6
Daily network	997.3	801.3	2167.6	14.1	8.3	28.5
Seven-day network	2297.5	977.7	3213.6	15.7	13.1	21.6
Selected meter	28.4	8.6	410.1	6.4	3.0	24.0
Completeness	1265.2	534.7	2667.4	16.0	7.1	25.8

Table 22: Dashboard refresh timings over ten measured refreshes. Aggregate cost from Section 6.1: 17.9 MB of storage (1.3 % of the raw table) and 55.7 s of initial refresh.

Judgement. The speed gain justifies the extra storage and refresh work:

- the whole dashboard is 6.7× faster, and the remaining 0.5 s is mostly browser rendering;
- individual panel queries are 4–150× faster;
- in Raw mode the five panels scan up to 2.7 million rows each at the same time and compete for CPU, and this cost grows with the date range and the number of users.

The price is small: 18 MB of storage, a 15-minute refresh job, and aggregates that may lag by up to about an hour unless real-time aggregation is enabled. All jobs were resumed after the measurements.

8 Forecast the next day

8.1 Prepare chronological data and features

Dataset.

- One row per region and CAT hour, summing the cleaned `energy_kwh`: 6,720 rows.
- Readings per region-hour ranged from 1,562 to 1,597 (1,600 planned).
- The data was reindexed onto a complete grid, which showed **no absent region-hours**. Rows with missing predictors would have been excluded.

Splits and forecast origin. The data was split by target timestamp, as shown in the second table below. The forecast origin is 00:00 CAT of the target's day.

Feature	Lag (h)	Window	Earliest usable target
lag_24h (same hour, previous day)	24	1 h	2 Sep 00:00
lag_48h	48	1 h	3 Sep 00:00
lag_168h (same hour, previous week)	168	1 h	8 Sep 00:00
prev_day_total (previous CAT day)	24–47	24 h	2 Sep 00:00
region, hour of day, day of week	calendar	–	–

Table 23: Features, all available before the forecast origin.

Split	Dates (targets)	Limits	Rows	Usable rows
Training	1–14 Sep	8 Sep 00:00 – 14 Sep 23:00	3,360	1,680
Validation	15–21 Sep	15 Sep 00:00 – 21 Sep 23:00	1,680	1,680
Test	22–28 Sep	22 Sep 00:00 – 28 Sep 23:00	1,680	1,680

Table 24: Chronological splits.

Lookback. The initial lookback is 168 hours. Of 3,360 training rows, 1,680 were excluded (targets before 8 September) and 1,680 were retained. The same rows were used for every configuration. We accepted this loss because the one-week lag and day of week capture the weekend dip and the Monday recovery.

Leakage check and preprocessing. The program checks that every lag source timestamp precedes its forecast origin; the smallest gap is 1 hour, so all sources do. Scaling and one-hot encoding were fitted on training rows only.

8.2 Evaluate the previous-day baseline

At 00:00 on each test day, each of the 24 hourly values for each region was predicted as the same hour on the previous day. This gave 1,680 predictions with **MAE 42.71 kWh** and **RMSE 79.40 kWh**.

8.3 Train and evaluate one regression model

We used Ridge regression. The shared predictors are the three lags, the previous-day total (standardised) and one-hot encodings of region, hour and day of week.

ID	Settings	Train rows	Validation rows	Val. MAE (kWh)	Val. RMSE (kWh)
R1	Ridge($\alpha = 0.1$)	1,680	1,680	**4.432**	5.643
R2	Ridge($\alpha = 1$)	1,680	1,680	4.440	5.652
R3	Ridge($\alpha = 10$)	1,680	1,680	4.818	6.183
R4	Ridge($\alpha = 100$)	1,680	1,680	7.955	11.773

Table 25: Validation comparison. R1 was selected and frozen.

The selected model was never refitted. It produced the same 1,680 test predictions as the baseline, using only information available before each origin; earlier completed test days supply later lag values only.

8.4 Judge forecast usefulness

Error is defined as predicted minus actual, in kWh.

Region	Baseline MAE	Model MAE	Baseline RMSE	Model RMSE	MAE gain
All (1,680)	**42.71**	**4.56**	**79.40**	**5.79**	**89.3 %**
Region 1	44.87	4.69	83.70	5.87	89.5 %
Region 2	41.63	4.53	76.81	5.65	89.1 %
Region 3	42.75	4.26	77.61	5.56	90.0 %
Region 4	34.86	4.06	64.23	5.12	88.4 %
Region 5	44.40	5.23	81.08	6.61	88.2 %
Region 6	39.43	3.72	74.77	4.69	90.6 %
Region 7	39.69	4.39	73.94	5.74	88.9 %
Region 8	55.97	5.82	102.06	7.17	89.6 %
Region 9	43.77	4.67	80.18	5.79	89.3 %
Region 10	39.72	4.23	74.28	5.30	89.4 %

Table 26: Test metrics in kWh, overall and by region.

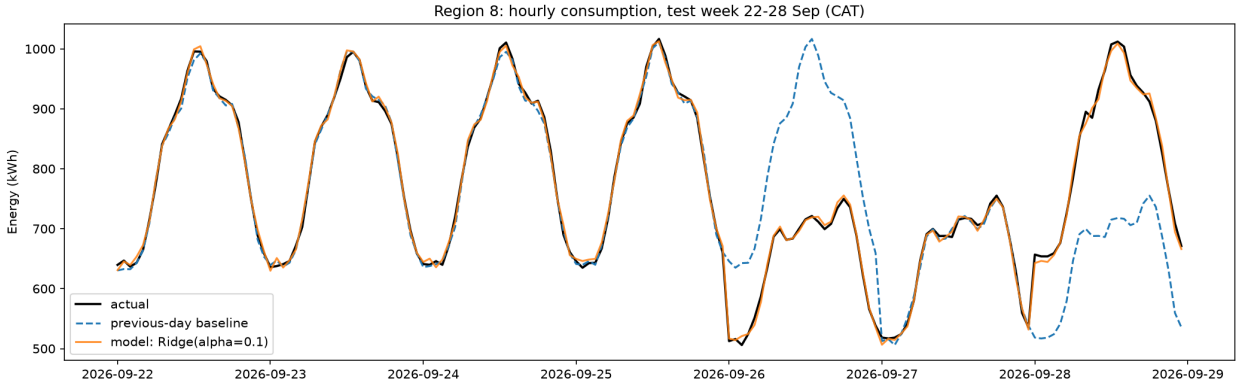


Figure 30: Region 8: actual, baseline and model hourly consumption over the seven test days.

Region	Peak hour (CAT)	Actual	Baseline	Abs. err.	Model	Abs. err.
Region 1	28 Sep 13:00	835.61	565.27	270.34	826.48	9.13
Region 2	28 Sep 13:00	780.13	546.91	233.21	774.24	5.89
Region 3	28 Sep 13:00	783.00	554.69	228.32	790.77	7.77
Region 4	28 Sep 18:00	659.90	561.41	98.50	655.98	3.92
Region 5	28 Sep 13:00	828.23	577.36	250.87	826.20	2.03
Region 6	28 Sep 13:00	749.11	504.41	244.70	731.38	17.73
Region 7	25 Sep 13:00	751.05	736.64	14.41	743.89	7.16
Region 8	25 Sep 13:00	1016.55	1010.53	6.02	1012.73	3.82
Region 9	23 Sep 13:00	808.96	796.89	12.07	804.44	4.52
Region 10	28 Sep 13:00	743.03	502.33	240.71	735.07	7.96

Table 27: Highest actual test hour per region (kWh); ties are broken by the earliest time.

Does the model improve on the baseline? Yes. The model improves on the baseline in every region, reducing MAE by about 89 %. On ordinary weekdays the two methods are similar (MAE about 5 kWh, under 1 % of a typical hourly value). The gain comes almost entirely from weekday–weekend transitions, where "yesterday" is the wrong comparison:

- the baseline's MAE was 135.7 kWh on Saturday 26 September and 139.5 kWh on Monday 28 September;
- the model's MAE was 3.6 and 4.9 kWh on those days.

Peak errors and usefulness for planning. The baseline under-forecast the Monday 13:00 peak by up to 270 kWh (32 %), which is the most dangerous error for capacity planning. The model's peak errors were 2–18 kWh (under 2.5 %), which is useful for day-ahead scheduling and procurement.

Limitations.

- The targets are observed totals that already miss about 1 % of readings, so the model forecasts recorded rather than true consumption.
- There was only one usable training week, and no holidays or weather data.
- In live use, the lags for the last hours before midnight would be incomplete because about 2 % of readings arrive up to three hours late (Section 5.3). The forecast should therefore run after about 03:00 or correct for provisional data.

9 Reproducibility and contributions

9.1 Code and repository

The repository (`https://github.com/<your-account>/<repository>`) and the submitted ZIP contain the same files:

- the unchanged generator;
- the capture, MQTT and ingestion programs (`src/`);
- the SQL files (`sql/00`–`10`);
- the benchmark, analysis and model code;
- the dashboard source and panel queries (`dashboard/`);
- the recorded timings, validation comparison and environment details (`results/`);
- a README with the execution order and commands.

The 1.16 GB CSV is not committed. `src/s2_1_capture_load.py` regenerates and reloads it. Passwords appear only as placeholders in `.env.example`.

9.2 Contributions

Student ID	Name	Contributions
101406	Pacifique Mashami	Environment setup, MQTT pilot and full ingestion pipeline (Sections 1–2), repository and README
101368	Prince Murenzi	Chunk-interval and compression benchmarks (Sections 3–4), continuous aggregates and freshness test (Section 6)
101371	Delice Sebaganwa Shami	Demand, regional, quality and anomaly analysis (Section 5), dashboards (Section 7), forecasting (Section 8)

Table 28: Group members and individual contributions. All members reviewed the full report and can explain every section.